

Cours — Node.js : Event Loop, Modules, Async/Await

Projet de référence : Chess Analytics Dashboard

Niveau : Étudiant S5 génie informatique

Style : Théorie + analogies + exemples de code commentés

Prérequis : Bases de JavaScript (variables, fonctions, objets)

Table des matières

1. C'est quoi Node.js, concrètement ?
 2. L'Event Loop — le cœur de Node.js
 3. Les modules — organiser son code
 4. Async / Await — gérer le temps d'attente
 5. Tout ensemble — un mini-serveur chess
 6. Résumé et points clés
-

1. C'est quoi Node.js, concrètement ?

Théorie

JavaScript a été inventé pour le navigateur — il permettait de rendre les pages web interactives. Pendant longtemps, impossible d'exécuter JavaScript en dehors d'un navigateur.

En 2009, Ryan Dahl a extrait le moteur JavaScript de Chrome (appelé **V8**) et l'a emballé dans un programme qu'on peut exécuter directement sur un système d'exploitation. C'est ça, **Node.js** : un *runtime* JavaScript côté serveur.

Runtime = environnement d'exécution. C'est le programme qui lit ton code JavaScript et l'exécute sur ta machine.

Analogie : JavaScript est une langue. Le navigateur est un interprète qui parle cette langue dans le contexte d'une page web. Node.js est un autre interprète qui parle la même langue, mais dans le contexte d'un serveur — il peut accéder aux fichiers, ouvrir des ports réseau, parler à une base de données.

Ce que Node.js ajoute à JavaScript pur

Dans un navigateur, JavaScript peut :

- manipuler le DOM (les éléments HTML)
- faire des requêtes HTTP (fetch)
- stocker des données localement (localStorage)

Dans Node.js, JavaScript peut en plus :

- **lire et écrire des fichiers** sur le disque
- **ouvrir un port réseau** et écouter des connexions TCP
- **parler à PostgreSQL** via un driver

- **appeler l'API Chess.com** depuis un serveur

Pourquoi Node.js dans ton projet

Dans ton Chess Dashboard, Node.js joue le rôle du backend. Il :

1. écoute sur un port (ex: 3000) les requêtes venant de Vue.js
2. parle à PostgreSQL pour lire/écrire les parties
3. appelle l'API Chess.com pour récupérer les parties brutes
4. renvoie du JSON formaté à Vue.js

2. L'Event Loop — le cœur de Node.js

Théorie

C'est le concept le plus important de Node.js, et le plus mal compris. On y va lentement.

Le problème que l'Event Loop résout

Imagine un serveur web classique qui doit gérer 1 000 utilisateurs en même temps. Chaque utilisateur fait une requête qui nécessite une requête en base de données (ça prend 50ms).

Approche classique (Java, PHP traditionnel) : créer un *thread* (fil d'exécution) par requête. 1 000 utilisateurs = 1 000 threads. Chaque thread attend sa réponse de la base de données en bloquant — il ne fait rien d'autre pendant 50ms. C'est du gaspillage de mémoire (chaque thread pèse ~1MB).

Approche Node.js : un seul thread principal. Quand une requête nécessite d'attendre (base de données, fichier, réseau), Node.js *délègue* l'attente au système d'exploitation et passe à autre chose. Quand la réponse arrive, il reprend le traitement. C'est **non-bloquant**.

Analogie : le serveur de restaurant.

Un serveur *bloquant* prendrait la commande de la table 1, irait en cuisine, attendrait debout que le plat soit prêt (5 minutes), reviendrait servir, puis passerait à la table 2. Avec 10 tables, il faudrait 10 serveurs.

Un serveur Node.js prend la commande de la table 1, dépose le bon en cuisine, passe à la table 2, prend sa commande, revient à la table 1 quand le plat est prêt. Un seul serveur pour 10 tables.

Comment ça marche exactement

Node.js tourne en permanence dans une boucle — l'**Event Loop** :

EVENT LOOP

1. Y a-t-il des événements en attente ?
 - requêtes HTTP entrantes
 - réponses de PostgreSQL
 - fichiers lus sur le disque
 - timers (setTimeout) expirés

2. Si oui → exécuter le callback associé
3. Si non → attendre
4. Recommencer

Les opérations lentes (I/O) sont confiées au système d'exploitation via **libuv** (une bibliothèque C++ intégrée à Node.js). Le thread principal continue de tourner — il ne bloque jamais sur de l'attente.

Ce qui bloque Node.js (à éviter)

Node.js est excellent pour les opérations I/O (réseau, fichiers, base de données). Il est mauvais pour les calculs CPU intensifs, parce que ceux-là ne peuvent pas être délégués — ils occupent le seul thread principal.

- ✓ Bon pour Node.js (opérations I/O – peuvent être déléguées)
 - Appel à l'API Chess.com (attente réseau)
 - Lecture en PostgreSQL (attente DB)
 - Lecture d'un fichier PGN sur le disque
- ✗ Mauvais pour Node.js (calcul CPU – bloque le thread)
 - Parser 100 000 parties PGN en une seule fois (calcul pur)
 - Chiffrer un mot de passe avec bcrypt de façon synchrone
 - Calculer des statistiques complexes sur 1M de lignes en JS

Dans ton projet, le parsing PGN de beaucoup de parties à la fois sera fait de façon asynchrone ou fragmentée pour cette raison.

Exemple commenté

```
// — Fichier : exemples/event_loop_demo.js —

console.log("1 – début du script");
// Node.js exécute cette ligne immédiatement, de façon synchrone

setTimeout(() => {
  // Cette fonction (callback) est enregistrée dans l'Event Loop
  // Elle sera exécutée dans ~2000ms, mais Node.js n'attend pas ici
  console.log("3 – dans le setTimeout (2 secondes plus tard)");
}, 2000);

console.log("2 – après le setTimeout");
// Cette ligne s'exécute IMMÉDIATEMENT après avoir enregistré le timer
// Node.js ne bloque pas en attendant les 2 secondes

// Résultat dans le terminal :
// 1 – début du script
// 2 – après le setTimeout
// 3 – dans le setTimeout (2 secondes plus tard)

// — Pourquoi c'est important pour ton projet —
```

```
// Quand Node.js appelle l'API Chess.com pour récupérer les parties,  
// il "enregistre" la requête et continue à traiter d'autres requêtes.  
// Quand Chess.com répond (200ms plus tard), l'Event Loop reprend.
```

3. Les modules — organiser son code

Théorie

Un module en Node.js, c'est simplement un fichier JavaScript. Au lieu de tout écrire dans un seul fichier de 5 000 lignes, on découpe le code en fichiers spécialisés qui s'importent entre eux.

C'est pour ça que ton projet aura `statsService.js`, `gameRepository.js`, `authController.js` etc. — chaque fichier est un module avec une responsabilité précise.

Il existe deux systèmes de modules en Node.js. Tu utiliseras le plus moderne : **ES Modules** (avec `import/export`).

Analogie : une bibliothèque universitaire. Chaque livre (module) traite un sujet précis. Quand tu rédiges un mémoire, tu *importes* les informations dont tu as besoin depuis plusieurs livres, plutôt que de tout recopier dans un seul document géant.

Les deux syntaxes (tu dois les reconnaître toutes les deux)

CommonJS — l'ancien système (encore très présent)

```
// — Exporter (dans le fichier qui partage) —  
const calculerWinrate = (victoires, total) => victoires / total;  
const formaterPourcentage = (ratio) => `${(ratio * 100).toFixed(1)}%`;  
  
module.exports = { calculerWinrate, formaterPourcentage };  
// On "exporte" un objet contenant les fonctions qu'on veut partager  
  
// — Importer (dans le fichier qui utilise) —  
const { calculerWinrate, formaterPourcentage } = require('./statsUtils');  
// require() cherche le fichier et retourne l'objet exporté
```

ES Modules — le système moderne (celui que tu utiliseras)

```
// — Exporter (dans le fichier qui partage) —  
export const calculerWinrate = (victoires, total) => victoires / total;  
export const formaterPourcentage = (ratio) => `${(ratio * 100).toFixed(1)}%`;  
// Le mot-clé "export" devant la déclaration suffit  
  
// On peut aussi exporter une valeur par défaut (une seule par fichier)  
export default class StatsService { /* ... */ }  
  
// — Importer (dans le fichier qui utilise) —
```

```
import { calculerWinrate, formaterPourcentage } from './statsUtils.js';
// Les accolades permettent de choisir ce qu'on importe

import StatsService from './statsService.js';
// Pas d'accolades pour un export default
```

Pour activer les ES Modules dans Node.js : ajoute `"type": "module"` dans `package.json`. Cursor le fera pour toi, mais maintenant tu sais pourquoi cette ligne existe.

Exemple concret — la structure de ton projet

```
// — Fichier : src/utils/chess.js —
// Module utilitaire : fonctions pures sur les données chess

// Calcule le winrate à partir d'un tableau de parties
export const calculerWinrate = (parties) => {
  if (parties.length === 0) return 0; // garde-fou : évite la
  division par zéro

  const victoires = parties.filter( // .filter() retourne un
    nouveau tableau
    (partie) => partie.resultat === 'win' // uniquement les parties
  ).length; // .length = nombre
  d'éléments

  return victoires / parties.length; // ratio entre 0 et 1
};

// Extrait l'ouverture d'un PGN brut
export const extraireOuverture = (pgn) => {
  const match = pgn.match(/\[Opening "(.+?)"\]/); // regex pour trouver le tag
  Opening
  return match ? match[1] : 'Inconnue'; // si trouvé, retourne le nom,
  sinon "Inconnue"
};
```

```
// — Fichier : src/services/statsService.js —
// Module service : logique métier des statistiques

import { calculerWinrate, extraireOuverture } from '../utils/chess.js';
// On importe les fonctions utilitaires depuis l'autre module

import { gameRepository } from '../repositories/gameRepository.js';
// On importe le repository qui parle à PostgreSQL

export const statsService = {

  // Retourne les stats complètes d'un joueur
```

```

async getStatsJoueur(username) {
  // Le service ne sait pas comment les données sont stockées
  // Il demande au repository, qui gère PostgreSQL
  const parties = await gameRepository.findByUsername(username);

  return {
    winrate: calculerWinrate(parties),          // utilise la fonction
    utilitaire
    totalParties: parties.length,
    topOuvertures: calculerTopOuvertures(parties) // fonction locale définie
    plus bas
  };
}
};

// Fonction privée (pas exportée) : visible uniquement dans ce fichier
const calculerTopOuvertures = (parties) => {
  const compteur = {};                          // objet vide pour compter

  parties.forEach((partie) => {
    const ouverture = extraireOuverture(partie.pgn);
    compteur[ouverture] = (compteur[ouverture] ?? 0) + 1;
    // ?? 0 : si compteur[ouverture] est undefined, on commence à 0
  });

  return Object.entries(compteur)                // convertit l'objet en tableau
    de paires [clé, valeur]
    .sort(([, a], [, b]) => b - a)                // trie par fréquence
    décroissante
    .slice(0, 10)                                // garde les 10 premières
    .map(([nom, count]) => ({ nom, count }));      // formate en objets propres
};

```

```

// — Fichier : src/controllers/statsController.js —
// Module controller : pont entre HTTP et les services

import { statsService } from '../services/statsService.js';

export const statsController = {

  // Appelé par Express quand GET /api/v1/stats/:username arrive
  async getStats(req, res, next) {
    try {
      const { username } = req.params;           // extrait "hikaru" de l'URL

      const stats = await statsService.getStatsJoueur(username);
      // appelle le service – le controller ne calcule rien lui-même

      res.status(200).json({ data: stats });      // répond avec le JSON formaté
    } catch (erreur) {
      next(erreur);                             // délègue l'erreur au
    }
  }
};

```

```
middleware centralisé
    }
  }
};
```

4. Async / Await — gérer le temps d'attente

Théorie

C'est le sujet qui déroute le plus les développeurs qui viennent de Java ou C. Voici pourquoi il existe.

Le problème : certaines opérations prennent du temps

Appeler l'API Chess.com prend ~200ms. Lire en PostgreSQL prend ~10-50ms. Pendant ce temps, le code ne peut pas "continuer normalement" — il doit attendre la réponse.

Il y a trois façons de gérer ça en JavaScript. Tu dois les connaître toutes les trois pour lire du code existant.

Étape 1 — Les callbacks (l'ancienne façon, à éviter)

```
// — Callbacks — l'ancienne façon —
// Problème : le "callback hell" — les imbrications deviennent illisibles

recupererArchives(username, function(erreur, archives) { // quand c'est prêt,
  appelle cette fonction
  if (erreur) {
    console.error('Erreur archives:', erreur);
  } else {
    parserPGN(archives, function(erreur2, parties) { // une autre fonction
      imbriquée
      if (erreur2) {
        console.error('Erreur parsing:', erreur2);
      } else {
        sauvegarderEnBase(parties, function(erreur3, resultat) { // encore une
          autre
          if (erreur3) {
            console.error('Erreur DB:', erreur3);
          } else {
            console.log('Import terminé:', resultat);
          }
          // Ici on est à 3 niveaux d'imbrication — c'est illisible
          // Imagine avec 5 ou 6 opérations...
        });
      }
    });
  }
});
```

Étape 2 — Les Promises (meilleur, mais encore verbeux)

Une **Promise** est un objet qui représente une valeur qui n'est pas encore disponible. C'est une "promesse" que la valeur arrivera (ou qu'une erreur se produira).

```
// — Promises – façon intermédiaire —

recupererArchives(username) // retourne une Promise
  .then((archives) => parserPGN(archives)) // .then() = quand c'est prêt
  .then((parties) => sauvegarderEnBase(parties)) // chaînable
  .then((resultat) => console.log('Import terminé:', resultat))
  .catch((erreur) => console.error('Erreur:', erreur)); // .catch() = si erreur
n'importe où

// C'est mieux, mais la gestion d'erreurs reste compliquée
// et ce n'est pas très naturel à lire
```

Étape 3 — Async/Await (ce que tu utiliseras toujours)

`async/await` est du "sucre syntaxique" — c'est une façon plus lisible d'écrire exactement la même chose que les Promises. En dessous, c'est la même mécanique.

- `async` devant une fonction signifie "cette fonction peut contenir des opérations asynchrones"
- `await` devant une Promise signifie "attends que cette Promise soit résolue avant de continuer"

Analogie : tu passes une commande en ligne. Sans `await`, tu continues à vivre ta vie et tu reçois une notification quand c'est livré (callback/Promise). Avec `await`, tu restes à la porte jusqu'à réception du colis. `async/await` fait ça *sans bloquer le thread* — Node.js s'occupe d'autres choses pendant l'attente, et revient quand c'est prêt.

```
// — Async/Await – la façon moderne (ce que tu vas écrire) —

// Le mot "async" est obligatoire pour utiliser "await" dans une fonction
async function importerPartiesJoueur(username) {

  // "await" pause l'exécution de CETTE FONCTION jusqu'à la réponse
  // Mais Node.js continue à traiter d'autres requêtes pendant ce temps
  const archives = await recupererArchives(username);
  // archives contient maintenant la valeur résolue de la Promise

  const parties = await parserPGN(archives);
  // même principe : on attend le résultat du parsing

  const resultat = await sauvegarderEnBase(parties);
  // même principe : on attend la confirmation de la base de données

  return resultat;
  // une fonction async retourne toujours une Promise
  // le return ici résout cette Promise avec "resultat"
```



```
}

// — La gestion d'erreurs avec try/catch —
async function importerAvecGestionErreurs(username) {
  try {
    // tout ce qui peut échouer est dans le try
    const archives = await recupererArchives(username);
    const parties = await parserPGN(archives);
    const resultat = await sauvegarderEnBase(parties);
    return resultat;

  } catch (erreur) {
    // si n'importe quelle ligne du try échoue, on arrive ici
    // erreur.message contient la description de l'erreur
    console.error(`Import échoué pour ${username}:`, erreur.message);
    throw erreur; // on re-lance l'erreur pour que l'appelant soit au courant
  }
}
```

Exemples concrets dans ton projet

Cas 1 — Vérifier qu'un joueur existe sur Chess.com

```
// — Fichier : src/integrations/chesscom/chesscomHttpClient.js —

export const chesscomHttpClient = {

  // Vérifie si un pseudo existe sur Chess.com (F-01)
  async verifierJoueur(username) {

    // fetch() est disponible nativement dans Node.js 18+
    // Il retourne une Promise – on utilise await pour attendre
    const reponse = await fetch(
      `https://api.chess.com/pub/player/${username}`,
      {
        headers: {
          // Chess.com recommande d'identifier ton application
          'User-Agent': 'ChessDashboard/1.0 contact@ton-email.com'
        }
      }
    );

    // reponse.status contient le code HTTP (200, 404, 429...)
    if (reponse.status === 404) {
      // Le joueur n'existe pas – on lance une erreur métier
      throw new Error(`Le joueur "${username}" n'existe pas sur Chess.com`);
    }

    if (reponse.status === 429) {
      // Trop de requêtes – Chess.com nous ralentit (NF-06)
      throw new Error('Rate limit Chess.com atteint, réessaie dans quelques
```

```
secondes');
}

if (!reponse.ok) {
  // reponse.ok = true si status entre 200 et 299
  throw new Error(`Erreur Chess.com: ${reponse.status}`);
}

// .json() est aussi asynchrone – on attend la désérialisation
const donnees = await reponse.json();

// On retourne seulement ce dont on a besoin (pas tout l'objet brut)
return {
  username: donnees.username,
  rating: donnees.stats?.chess_blitz?.last?.rating ?? null,
  // ?. = optional chaining : évite le crash si chess_blitz est undefined
  // ?? null = si undefined, retourne null
};
}
};
```

Cas 2 — Récupérer les archives de parties (avec await multiple)

```
// — Fichier : src/services/importService.js —

import { chesscomHttpClient } from
'../integrations/chesscom/chesscomHttpClient.js';
import { gameRepository } from '../repositories/gameRepository.js';

export const importService = {

  // Importe toutes les parties d'un joueur (F-02)
  async importerHistorique(username) {

    // Étape 1 : récupérer la liste des archives (une URL par mois)
    const reponseArchives = await fetch(
      `https://api.chess.com/pub/player/${username}/games/archives`
    );
    const { archives } = await reponseArchives.json();
    // archives = ["https://api.chess.com/.../2024/01", "https://.../2024/02",
    ...]

    let totalImportees = 0;

    // Étape 2 : pour chaque mois, récupérer les parties
    // On utilise une boucle for...of (compatible avec await)
    // ATTENTION : forEach() ne fonctionne PAS avec await – c'est un piège
    classique
    for (const urlArchive of archives) {

      // Vérifier si ce mois est déjà en base (NF-01 : cache)
```

```
const moisDejaImporte = await gameRepository.moisExisteEnBase(
  username,
  urlArchive
);

if (moisDejaImporte) {
  continue; // passe au mois suivant sans appeler Chess.com
}

// Récupérer les parties de ce mois
const reponse = await fetch(urlArchive);
const { games } = await reponse.json();

// Sauvegarder en base
await gameRepository.insererParties(username, games);
totalImportees += games.length;
}

return { totalImportees };
}
};
```

Cas 3 — Le piège classique : forEach avec async/await

```
// — Ce que BEAUCOUP de développeurs font (FAUX) —

const archives = ['url1', 'url2', 'url3'];

// ❌ FAUX : forEach ignore les Promises retournées par async
// Node.js lance toutes les requêtes en même temps et n'attend pas
archives.forEach(async (url) => {
  const data = await fetch(url); // ← await ignoré par forEach
  await sauvegarder(data);       // ← peut s'exécuter dans n'importe quel
  ordre
});
// Code après le forEach s'exécute IMMÉDIATEMENT, avant même que les fetches
soient terminés

// — La solution 1 : for...of (séquentiel – l'un après l'autre) —
// Utilise ça quand l'ordre compte ou que tu veux éviter de surcharger Chess.com

for (const url of archives) {
  const data = await fetch(url); // ← attend vraiment
  await sauvegarder(data);       // ← s'exécute après le fetch
}
// Simple, lisible, prévisible

// — La solution 2 : Promise.all (parallèle – tout en même temps) —
// Utilise ça quand l'ordre n'a pas d'importance et que tu veux aller vite
```

```
// ATTENTION avec Chess.com : trop de requêtes parallèles = rate limit (429)

const resultats = await Promise.all(
  archives.map(async (url) => { // .map() crée un tableau de Promises
    const data = await fetch(url);
    return data.json();
  })
);
// resultats = tableau avec tous les résultats dans le même ordre que archives
```

5. Tout ensemble — un mini-serveur chess

Voici un exemple complet qui relie tous les concepts vus. C'est une version simplifiée de ce que tu vas construire.

```
// — Fichier : src/index.js – Point d'entrée de l'application —

import { app } from './app.js'; // importe l'app Express configurée
import { config } from './config/env.js'; // importe la configuration

const PORT = config.PORT ?? 3000; // ?? : utilise 3000 si PORT n'est pas défini

// Démarre le serveur et écoute sur le port
// app.listen() retourne une Promise – on utilise await
const serveur = app.listen(PORT, () => {
  // Ce callback s'exécute quand le serveur est prêt
  console.log(`✅ Serveur démarré sur http://localhost:${PORT}`);
});

// Gestion propre de l'arrêt (Ctrl+C ou signal du système)
process.on('SIGTERM', () => {
  console.log('🛑 Arrêt du serveur...');
  serveur.close(() => {
    console.log('✅ Serveur arrêté proprement');
    process.exit(0);
  });
});
```

```
// — Fichier : src/app.js – Configuration Express —
// Séparé de index.js pour permettre les tests sans démarrer le réseau

import express from 'express'; // framework web
import { statsRouter } from './api/v1/routes/stats.routes.js';

export const app = express(); // crée l'application Express
```

```
// — Middlewares globaux —
// Ces fonctions s'exécutent pour TOUTES les requêtes, dans l'ordre

app.use(express.json());
// Permet à Express de lire le body JSON des requêtes POST/PUT
// Sans ça, req.body serait undefined

app.use((req, res, next) => {
  // Middleware de logging simple (tu utiliseras Morgan ou Winston en vrai)
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);
  next(); // OBLIGATOIRE : passe au middleware suivant
});

// — Routes —
app.use('/api/v1/stats', statsRouter);
// Toutes les requêtes vers /api/v1/stats/* sont gérées par statsRouter

// Route de santé (NF-12 : Docker healthcheck)
app.get('/health', (req, res) => {
  res.status(200).json({ status: 'ok', timestamp: new Date().toISOString() });
});

// — Middleware de gestion d'erreurs —
// DOIT être déclaré EN DERNIER – 4 paramètres obligatoires (NF-09)
app.use((erreur, req, res, next) => {
  console.error('Erreur non gérée:', erreur);

  // Format JSON uniforme pour toutes les erreurs (NF-09)
  res.status(erreur.status ?? 500).json({
    error: {
      code: erreur.code ?? 'ERREUR_INTERNE',
      message: erreur.message ?? 'Une erreur est survenue',
      requestId: req.headers['x-request-id'] ?? 'inconnu'
    }
  });
});
```

```
// — Fichier : src/api/v1/routes/stats.routes.js —

import { Router } from 'express';
import { statsController } from '../../../controllers/statsController.js';

export const statsRouter = Router(); // crée un routeur Express

// GET /api/v1/stats/:username
// :username est un paramètre dynamique – accessible via req.params.username
statsRouter.get('/:username', statsController.getStats);

// GET /api/v1/stats/:username/openings
statsRouter.get('/:username/openings', statsController.getOpenings);
```

```
// — Fichier : src/controllers/statsController.js —

import { statsService } from '../services/statsService.js';

export const statsController = {

  // Gère GET /api/v1/stats/:username
  async getStats(req, res, next) {
    try {
      const { username } = req.params;      // extrait le paramètre de l'URL
      const { format } = req.query;        // extrait ?format=blitz de l'URL

      // Validation simple (tu utiliseras Zod ou Joi en vrai)
      if (!username || username.length > 50) {
        return res.status(400).json({      // 400 = Bad Request
          error: { code: 'USERNAME_INVALIDE', message: 'Username invalide' }
        });
      }

      // Appel au service – le controller ne sait pas d'où viennent les données
      const stats = await statsService.getStatsJoueur(username, { format });

      res.status(200).json({ data: stats }); // 200 = OK

    } catch (erreur) {
      next(erreur); // délègue à errorHandler.js (dernier middleware de app.js)
    }
  },

  async getOpenings(req, res, next) {
    try {
      const { username } = req.params;
      const openings = await statsService.getTopOuvertures(username);
      res.status(200).json({ data: openings });
    } catch (erreur) {
      next(erreur);
    }
  }
};
```

6. Résumé et points clés

Ce que tu dois retenir

Concept	En une phrase	Dans ton projet
Node.js	Runtime JavaScript côté serveur, basé sur V8	Ton backend — lit les requêtes Vue.js, parle à PostgreSQL et Chess.com

Concept	En une phrase	Dans ton projet
Event Loop	Boucle unique non-bloquante qui gère tous les événements	Permet à ton serveur de gérer plusieurs requêtes simultanées sans thread par requête
Module	Fichier JavaScript qui exporte et importe des fonctionnalités	<code>statsService.js</code> , <code>gameRepository.js</code> , <code>statsController.js</code> ...
Promise	Valeur future — représente une opération qui n'est pas encore terminée	Retournée par <code>fetch()</code> , <code>db.query()</code> , etc.
async/await	Façon lisible d'attendre les Promises sans bloquer le thread	Utilisé partout : dans les services, repositories, controllers

Les pièges à ne pas reproduire

```
// ❌ Piège 1 : forEach avec async (les await sont ignorés)
tableau.forEach(async (item) => { await faireTruc(item); });

// ✅ Solution
for (const item of tableau) { await faireTruc(item); }

// ❌ Piège 2 : oublier await (on travaille avec la Promise, pas la valeur)
const parties = gameRepository.findAll(); // parties est une Promise, pas un tableau !
console.log(parties.length); // undefined – la Promise n'a pas de .length

// ✅ Solution
const parties = await gameRepository.findAll(); // parties est maintenant un tableau
console.log(parties.length); // fonctionne

// ❌ Piège 3 : await en dehors d'une fonction async
const data = await fetch('...'); // SyntaxError : await is only valid inside async functions

// ✅ Solution : wrapper dans une fonction async
async function chargerDonnees() {
  const data = await fetch('...');
  return data;
}

// ❌ Piège 4 : ne pas gérer les erreurs
const data = await fetch('https://api.chess.com/...'); // si réseau coupé, crash non géré

// ✅ Solution : try/catch
try {
```

```
const data = await fetch('https://api.chess.com/...');
} catch (erreur) {
  console.error('Échec de la requête Chess.com:', erreur.message);
}
```

Questions de validation

Avant de passer au module suivant, assure-toi de pouvoir répondre à ces questions sans regarder le cours :

1. Quelle est la différence entre Node.js et JavaScript dans un navigateur ?
2. Pourquoi Node.js peut gérer 1 000 connexions avec un seul thread ?
3. Dans ton projet, quelle couche utilise `import/export` pour partager du code ?
4. Pourquoi `forEach` ne fonctionne pas avec `async/await` ?
5. Qu'est-ce qu'une Promise et pourquoi `async/await` la rend plus lisible ?

Pour aller plus loin (optionnel)

Si une notion reste floue après ce cours, voici les ressources ciblées :

- **Event Loop visuellement** : cherche "What the heck is the event loop anyway?" — Philip Roberts (JSConf 2014) — la meilleure explication visuelle qui existe, 26 minutes
- **Async/Await** : cherche "Fireship async await" — 7 minutes, très dense
- **Documentation officielle Node.js** : nodejs.org/en/docs/guides/event-loop-timers-and-nexttick

Règle : si tu bloques sur une notion spécifique de ce cours, note la question précise et pose-la — soit à Cursor pour un exemple de code, soit ici pour la théorie.